When the Backtest Lies: Twenty-One Data and Engine Errors from a Gold Trading Research Pipeline

Oleksandr Sytnyk

October 2026

Abstract

This note documents twenty-one errors found during 2026 in a research pipeline for intraday gold strategies: an event-driven Python backtest engine fed by a MetaTrader 5 broker, Dukascopy and CME futures data. None of them crashed anything or produced an obvious warning. Each produced plausible numbers, and several reversed a conclusion: a strategy with a 79% win rate and a profit factor of 12 had no edge at all; another went from profit to loss when one data feed was replaced by another; a month that seemed to break a strategy was an hour of misaligned clocks; a profitable trade around US data releases was rejected because its window was set in UTC; and a plan to move to futures rested on a spread estimate three years out of date. For each error the note gives what happened, how it was found and how much it moved the result, and it closes with the checks that caught them. Errors that appear only when a backtest is compared with live trading are covered in a companion note.

AI disclosure. The author used Claude (Anthropic), an AI assistant, to write the data-analysis code, to draft and edit the text, to assist with wording and structure, and as a consultative tool for discussing methodological and technical questions. The research questions and the decisions on what to test and what to accept are the author's. The author reviewed, verified, and edited all AI-assisted content and takes full responsibility for the content of the paper, including its analysis, results, and conclusions.

1 Why this note

Research on short-horizon effects, where a typical trade earns a few basis points, is more exposed to data errors than to bad ideas. An hour's shift in timestamps, a few synthetic bars or a commission charged 64 times over can be larger than the effect being measured. The errors below were all found in one pipeline between May and October 2026, while developing the strategies described in two companion notes on the London gold fix and the weekend close. They are grouped by where they occur: the clock, the price and cost data, silent truncation of files, the backtest engine and the method of analysis.

Exhibit 1. The twenty-one errors

#	Error	How it was found	Effect
Clock			
1	Broker daylight-saving dates taken from Europe, not the US	strategy's worst month did not appear in futures data	4 weeks a year an hour late; a losing month was profitable
2	Time offset fixed at midnight on the day of a clock change	conflicts when merging data	30 minutes of prices about $50 away from the market
3	Event window fixed in UTC instead of New York time	idea rerun under one protocol	a working strategy rejected; window an hour late 8 months a year
Price and cost data			
4	Futures ticks switch contracts in the middle of a trade	largest trades checked against contract prices	$33,800 of a $94,000 result was false
5	21.8% of one feed's bars are synthetic	bars per day counted	sign of a strategy's result reversed
6	Data outages taken as market closures	dates of a small group listed	46 "holidays" were 18
7	Futures spread assumed at one tick	spread measured from trades	+130% on the CFD, −99.8% on futures
8	Order book rebuilt without a starting snapshot	spread distribution checked	crossed book, spreads of −$112
9	Unsigned trade sizes negated	sign check on a known relationship	all order flow read as buying
Silent truncation			
10	30 GB export ends in zero bytes three years early	last timestamp of each file	study cut to 2023 without warning
11	Tick cache: empty, partial or stale days kept	"strategy stopped trading"	backtests ended days early; A/B tests on different data
Engine			
12	Missing symbol falls back to a default commission	per-trade cost column	commission 64 times too high
13	Limit orders filled at prices the market never reached	result too good to be true	79% win rate, profit factor 12, no real edge
14	Trend filter shorter than its own buffer	two runs identical to the cent	filter silently did nothing
15	Strategy never told of take-profit exits	positions left as zombies	entries blocked; daily limits blind to wins
Method			
16	Higher-timeframe state built from the full future bar	honest recalculation	a filter's conclusion reversed
17	Stops and targets resolved on one-hour bars	re-resolved on one-minute bars	+0.16 R per trade became −0.02 R
18	Compounding inflates late losses	same trade under flat sizing	a $45 loss shown as $10,072
19	Overlapping forecast windows	non-overlapping recount	$t = 21.6$ became 2.9, then 1.2 out of sample
20	Parameters fitted on the most volatile months	five-year rerun	profit factor 2.0 became 1.2
21	Splitting past trades taken as a filter's effect	filter built into the engine	profit factor 0.85 vs 1.16 between groups; in the engine 1.03 to 1.05

2 The clock

1. The broker changes its clocks on US dates

What happened. The broker's server runs on New York time plus seven hours, so it moves to summer time on the US dates, the second Sunday of March and the first Sunday of November. The pipeline used the European dates, the last Sundays of March and October. For about three weeks in spring and one in autumn every broker timestamp was therefore an hour late. Any strategy that trades at a fixed time of day, in the backtest and in live trading, traded an hour away from its intended time in those weeks.
How it was found. In the backtest on the broker's data, a strategy that sells gold into the London morning fix lost 14 of 22 trades to the stop in March 2026, while the same window in CME gold futures gained 103 basis points that month. Comparing the move from 10:24 to 10:30 London time on the broker and on the exchange day by day gave a correlation of 1.00 outside the affected weeks; inside them, the broker's move matched the exchange's an hour later. An earlier check, that the trading week always opens at 01:00 server time, had passed: in the affected weeks the shifted opening looks exactly like the normal winter one.
Effect. After the fix and a fresh download of the affected weeks, March 2026 for that strategy changed from a loss to a gain of \$3,029 per lot, its maximum drawdown fell from \$5,300 to \$2,100 per lot, and an earlier conclusion that entering one minute later was safer turned out to be an artefact of the false March. Every backtest that included those weeks had to be rerun.

2. The offset was fixed at midnight on the day of the change

What happened. On the day of a clock change the offset was taken at the start of the day, while the change happens later that day, before the Sunday evening open. In spring the Sunday evening ticks were stamped an hour late and landed on top of Monday's first hour. The bar builder kept the first of each pair of duplicates, so 30 minutes of Monday 30 March 2026 carried prices about \$50 away from the market.
How it was found. Merging new data reported 33 conflicts with existing bars. Checking the opening time of every week in the cache showed exactly two exceptions, both on days of a clock change. The live connector had the same problem in another form: it measured the server offset once at start-up, so a bot running through a clock change would have mislabelled every later candle.
Effect. A \$50 gap inside a session is worse than a missing day: a missing day is visible, while a gap looks like a market move and triggers stops and breakouts.

3. An event window fixed in UTC

What happened. A straddle around US data releases was tested with its window fixed at 13:30 UTC. The data were stamped correctly; the error was in the analysis. US releases come out at 08:30 New York time, which is 13:30 UTC only in winter; from the second Sunday of March to the first Sunday of November it is 12:30 UTC. For eight months of every year the test measured the hour after the release.
How it was found. The idea was rerun, months later, through a common protocol in which every event window is defined in the time zone of the event itself. The window code of the old test then showed the fixed UTC time.
Effect. The idea had been rejected: every variant lost money. At the correct time it earned $+0.20\,\mathrm{R}$ per trade after costs over 16 years of futures, $t = 5.1$, and $+0.51\,\mathrm{R}$ per trade on the broker's ticks. Errors 1 and 2 shifted the data; this one shifted only the question, and was just as costly.

3 Price and cost data

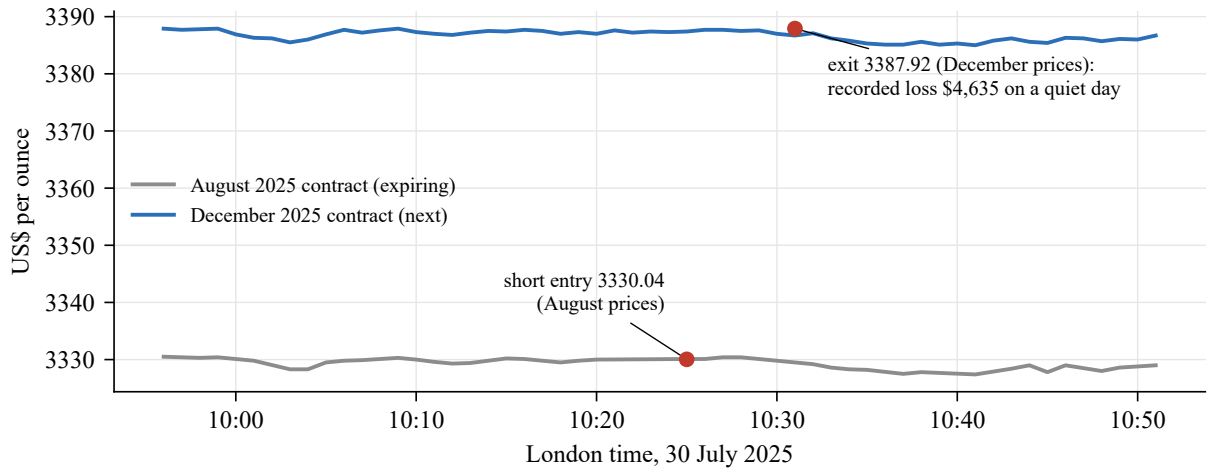
4. Futures ticks switch contracts in the middle of a trade

What happened. A continuous futures series has to switch from the expiring contract to the next one. In the tick data exported for the MetaTrader 5 Strategy Tester the switch was not fixed to a day: on roll days the ticks jumped between the two contracts, while the one-minute bars of the same instrument had already moved to the new contract. On 30 July 2025 the next gold contract traded about $57 above the expiring one (Exhibit 2).

How it was found. In a 16-year test of the London fix strategy as a MetaTrader 5 Expert Advisor, the largest trades were listed and checked against the prices of individual contracts. Twenty-seven trades on roll days had moved $15–60 in six minutes, far more than either contract moved in those minutes.

Effect. These 27 trades contributed $33,800 of the test's $94,000. Without them the result was $60,200, the maximum drawdown deepened from $6,100 to $7,900, and 2024 changed from a gain of $9,700 to a loss of $1,400.

Exhibit 2. A recorded loss created by a contract roll, 30 July 2025



Notes:
- Lines: one-minute closes of the August and December 2025 gold futures contracts.
- Points: entry and exit prices of a six-minute short in the MetaTrader 5 tick test, which entered on the August contract and exited on the December one.

Source: CME GC futures by contract, Databento; MetaTrader 5 Strategy Tester report.

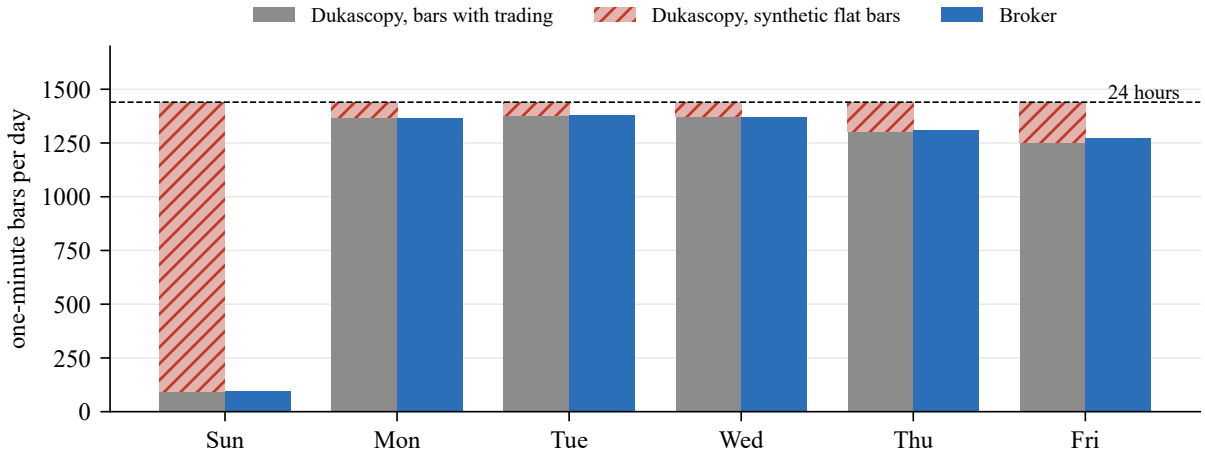
5. A fifth of one feed's bars are synthetic

What happened. The Dukascopy gold feed fills every minute of every day, including Sundays when the market is closed, by repeating the last price with no volume. Over August 2025 to May 2026, 74,649 of its 342,720 one-minute bars (21.8%) have the same high and low and zero volume, including 93.6% of Sunday bars (Exhibit 3). The broker's feed over the same months has one such bar.

How it was found. Two feeds were compared over the same window after strategies kept doing worse on Dukascopy. The median number of bars per day was 1,440, a full 24 hours, which gold does not trade.

Effect. The frozen weekend price followed by the live Monday price looks like a sharp move. A dip-buying strategy entered on it: over the same period it made 634 trades with a profit factor of 1.13 and +$2,679 on the broker's data, and 585 trades with a profit factor of 0.73 and −$1,495 on Dukascopy. A trend filter that looked useful on Dukascopy, because it blocked the post-weekend entries, cost $3,090 on the broker's data.

Exhibit 3. One-minute bars per day by weekday, Dukascopy and broker feeds, August 2025 – May 2026



Notes:
– Synthetic bar: high equal to low and zero volume.
– The broker's feed opens on Sunday evening; the market is closed on Saturdays and no feed has bars then.
Source: Dukascopy and MetaTrader 5 broker one-minute XAUUSD data.

6. Data outages taken as market closures

What happened. A test of how gold moves before market closures defined a closure as any gap of at least 45 minutes in one-minute futures data. Interruptions in the data feed also create such gaps.
How it was found. Listing the dates in the small "weekday holiday" group showed ordinary days such as 26 January and 3 February, where the data stopped at 19:00 or 20:17 and resumed the next day. Real closures end at the scheduled reopening, 18:00 New York time; outages end at random times.
Effect. The group shrank from 46 days to 18, its average move from +4.3 to +2.7 basis points and its t-statistic from 2.5 to 1.8, and in silver the effect disappeared. A finding that had looked like a clean step in the mechanism became too thin to report.

7. The futures spread was assumed, not measured

What happened. Every cost estimate for gold futures used a spread of one tick, $0.10 per ounce, taken from the contract specification. That was true in 2023, when the best quote held six to eight contracts. By 2026 the quoted spread had widened to $0.345, the spread actually paid by trades to $0.463, and the depth at the best price had fallen to one or two contracts; only 18% of volume traded at a one-tick spread.
How it was found. Thirty million trades with the prevailing best bid and offer over a year were downloaded to test an order-book idea, and the spread was measured from them as a by-product.

Effect. On the same bars, a dip-buying strategy returned −37.7% with the assumed spread and −99.8% with the measured one, against +130% on the broker's CFD at its measured $0.114. A plan to move from the CFD to futures because futures were cheaper had rested entirely on the assumed number; futures turned out four times more expensive.

8. An order book rebuilt without its starting state

What happened. A full order book was rebuilt by replaying every add, modify and cancel message of the day. The data feed delivers changes only, without a snapshot of the orders already resting at the start, so orders placed before the window were never removed and accumulated all day.
How it was found. The rebuilt book's spread was checked before using it: the median looked plausible at $0.40, but the 90th percentile was $18.70 and some spreads were negative, down to −$112, a crossed book.
Effect. Every order-book feature built on it would have been noise. The study switched to a data schema in which the best bid and offer are already computed.

9. Unsigned trade sizes

What happened. Trade sizes in the futures trade file were stored as unsigned 32-bit integers. Selling volume was computed by negating the size, which for an unsigned integer gives 2^{32} minus the size, a huge positive number. All order flow became buying.
How it was found. A control test: buying pressure and the price change in the same minute must be positively related. The relationship came out negative, and the cause was traced to the data type.
Effect. Two full runs of an order-flow study were wasted before the control caught it. Without the control, a nonsensical result could have been interpreted as a contrarian signal.

4 Silent truncation

10. A 30 GB file that ends three years early

What happened. A compressed one-second export of CME equity index futures, labelled as running to September 2026, ended in April 2023. Past that point the file contained zero bytes; decompression and parsing completed without an error.
How it was found. The first and last timestamp of every file were printed and compared with the file names. Per-contract S&P 500 futures ended on 17 April 2023.
Effect. An options study that needed per-contract futures prices lost its last three years and had to state that limit; a reader of the summary statistics alone would not have known.

11. Four ways a tick cache ends early without saying so

What happened. In August 2026 backtests stopped producing trades after 13 August. No strategy was at fault. Four separate faults in the data cache each cut the data short:

- an empty file was saved when a day returned no ticks, for example when it was requested before it had happened, and the day was never requested again;
- a day downloaded while still in progress was never completed: 13 August held 219,924 ticks instead of 669,831;

- the repair tool did not remove index entries for deleted days, so the integrity check failed forever;
- the cache key for built candles did not include the data source, so a rerun after new data arrived silently reused the old version.

How it was found. The log printed the requested date range rather than the range actually loaded, which sent the first diagnosis towards the strategies. Printing the first and last bar actually used found it at once.
Effect. Besides shortened backtests, the last fault is the most dangerous for research: "before" and "after" in an A/B comparison can be computed on different data.

5 The backtest engine

12. A missing symbol falls back to a crypto commission

What happened. The Dukascopy gold symbol was missing from the instrument configuration. The engine fell back to its default, a percentage-of-turnover commission with a contract size of one, built for crypto. A trade of 0.25 lots was charged about $96 instead of $1.50. A command-line override did not help, because it was applied only when the symbol's configuration existed.
How it was found. The commission column of individual trades was compared with the broker's rate of $6 per lot.
Effect. Every backtest on that feed carried costs about 64 times too high. Results on Dukascopy had been consistently worse than on the broker's feed, and the difference had been attributed to the feeds themselves.

13. Limit orders filled where the market never traded

What happened. The engine decided how to fill a pending limit order by checking whether its side was "BUY". Orders in the engine are labelled "LONG" and "SHORT", so every buy limit fell through to the sell rule: fill when the bar's high reaches the limit. For a buy limit below the market that is always true. The order was filled on the next bar at the limit price, below that bar's actual low, and its take-profit was then hit within the same minute.
How it was found. The strategy showed a 79% win rate and a profit factor of 12, and 49% of its trades closed in the minute they opened. Questions about whether this was a bug led to a filled bar whose low, 4,725.32, was above the limit price of 4,716.27 at which the order had been filled.
Effect. After the one-line fix, the share of trades winning in their first minute fell from 98% to 7%, and both sides of the strategy lost money (profit factor 0.82 for longs, 0.63 for shorts). It had never been traded live: a broker fills a limit only when the price is reached.

14. A trend filter shorter than its own memory

What happened. A strategy kept its recent closes in a fixed-length buffer. A new 200-period moving-average filter needed more closes than the buffer held, so the average could never be computed and the filter let every trade through.
How it was found. The backtest with the filter switched on matched the one without it exactly: the same 89 trades, +59.14%, profit factor 1.46. Identical results to the cent after a change are a bug report.

Effect. None in money, because it was caught at once; but a filter that silently does nothing would have been reported as "tested, no improvement" and discarded. The buffer is now sized from the longest period any indicator needs.

15. The strategy was never told its trade had hit the target

What happened. To avoid notifying a strategy about partial exits, the engine skipped the notification for every exit labelled "first take-profit". A position with a single target closes in full under that same label, so the strategy never learned that its positions had closed at the target.
How it was found. While adding pyramiding to a trend strategy, baskets that had closed at their target stayed open in the strategy's own records and blocked new entries.
Effect. Any strategy logic that depends on closed trades, such as daily loss limits or position counters, was blind to winning trades. Historical results were not distorted only because those limits happened to be off.

6 The method of analysis

16. The hourly state knew the rest of the hour

What happened. To filter five-minute signals by the state of the hourly chart, five-minute bars were resampled into hours. Each signal was then compared with the completed hour, including the five-minute bars that came after it.
How it was found. Recomputing the hourly state from the part of the hour that had already happened.
Effect. The conclusion reversed. With look-ahead, five-minute signals without confirmation from the hourly chart won 56.4% of the time and looked like the ones to trade. Computed honestly, they won 49.5%, and the signals confirmed by the hourly chart, the ones the look-ahead version would have discarded, won 57.5%.

17. One-hour bars hide the order of stops and targets

What happened. A pattern traded on one-hour bars had its stop and target about half to three quarters of an hour's average range away from the entry. When both lie inside the same bar, a test on hourly highs and lows cannot tell which was hit first.
How it was found. The same trades were resolved on one-minute bars.
Effect. The average result fell from $+0.16\,\mathrm{R}$ to $-0.02\,\mathrm{R}$ per trade, and years with no loss became mostly losing years. Any test with tight stops is now resolved on one-minute or tick data.

18. Compounding makes late losses look like disasters

What happened. In a backtest that reinvests profits, position size grows with the account, so a loss late in the test can be hundreds of times larger in dollars than the same loss early on.
How it was found. A single trade shown as a loss of \$10,072 started a search for its cause. Under flat sizing the same trade lost \$45, 224 times less. A pattern of "losing Wednesdays" had the same origin: measured as a percentage of the balance, every weekday was positive.
Effect. An optimisation was started to fix a problem that did not exist. Strategies are now judged in R or under flat sizing first, and compounding is used only to project returns.

19. Overlapping windows multiply the evidence

What happened. A test of whether the order book predicts the next 60 minutes used one observation per minute. Each 60-minute outcome overlaps the next 59, so consecutive observations are nearly the same number counted again, and the usual t-statistic treats them as independent.
How it was found. The largest t-statistic of the study, 21.6 for the spread predicting the next hour, was recomputed on non-overlapping hours.
Effect. It fell to 2.9 in the sample used for discovery and to 1.2 on later data. A related lesson from the same work: a sign that is stable in four years out of four is weak evidence for slow signals, because a year holds only about a thousand independent hours and neighbouring years share a market regime.

20. Parameters fitted on the most volatile months

What happened. Strategies were tuned on a rolling window of the last four to six months. In spring 2026 that window covered the most volatile period in the data: annualised gold volatility near 39% against about 19% in the first half of 2025, and US-session ranges three times wider.
How it was found. A trend strategy tuned to a profit factor of 2.0 on 159 days was rerun over five years.
Effect. Over five years its profit factor was about 1.2. Parameters fitted to an unusual regime look like an edge until the regime ends; the remedy is to test on a period with a different regime before accepting them.

21. Splitting past trades is not testing a filter

What happened. A candidate filter was evaluated by labelling a strategy's past trades with the filter's value and comparing the groups: trades taken in smooth trends had a profit factor of 0.85, trades in choppy markets 1.16. That looks like a filter worth adding.
How it was found. The filter was built into the strategy and the backtest rerun.
Effect. The profit factor moved from 1.03 to 1.05 on one feed and from 1.63 to 1.60 on the other: nothing. A strategy with cooldowns, daily limits and one position at a time does not simply lose the filtered trades; it takes different ones in their place. Any filter found by splitting past trades has to be confirmed inside the engine.

7 The checks that caught them

Most of the twenty-one errors were found by a few habits rather than by careful reading of code:

- **Define every time in the clock of the thing it describes.** Releases follow New York, the London fix follows London, the broker follows its server; a fixed UTC time drifts against all of them (error 3).
- **Two independent sources, minute by minute.** Comparing the broker with exchange futures and one feed with another found errors 1, 4 and 5.
- **Measure inputs that are usually assumed.** Spreads, commissions and contract sizes come from configuration or memory; measuring them from trades found errors 7 and 12.
- **Count bars per day and print the first and last timestamp of every file.** This finds synthetic data, outages and truncation (errors 5, 6, 10 and 11) in seconds.

- **Log the data actually used, not the range requested** (error 11).
- **Run a control whose answer is known.** A relationship that must be positive, or a book that must not be crossed, caught errors 8 and 9.
- **Treat a result that is too good, or too identical, as a bug report.** A 79% win rate with trades that win in their first minute (error 13), or two runs that agree to the cent after a change (error 14).
- **Resolve stops on the finest data available, and compute any higher-timeframe state only from the past** (errors 16 and 17).
- **Judge in R or under flat sizing before compounding** (error 18), **count independent observations, not rows** (error 19), and **test on a different regime from the one used for fitting** (error 20).
- **Confirm every filter inside the engine** (error 21), and **list the actual dates in any small group** before interpreting its average (error 6).

One rule follows from all of them: after fixing an error, rerun everything that depended on the affected data. Error 1 alone changed the March results of every time-of-day strategy in the pipeline, and an earlier conclusion built on the false month survived until it was explicitly retested.

8 Conclusion

None of these errors was exotic. Clock changes, contract rolls, filler bars, stale cost assumptions, truncated files, default configurations and look-ahead are known problems; what made them costly was that each produced numbers that looked like ordinary market behaviour. In a pipeline that measures effects of a few basis points, data checks are part of the research itself, and a result is not finished until it has survived a second source, a count of its inputs and a test of its assumptions.